

SAMSUNG PRISM · GENERATIVE AI HACKATHON · 3RD
EDITION

Streaming Live RAG — Architecture Brief

Team Nexora

SRM Institute of Science and Technology, Chennai

October 2026

SL-RAG Architecture Brief

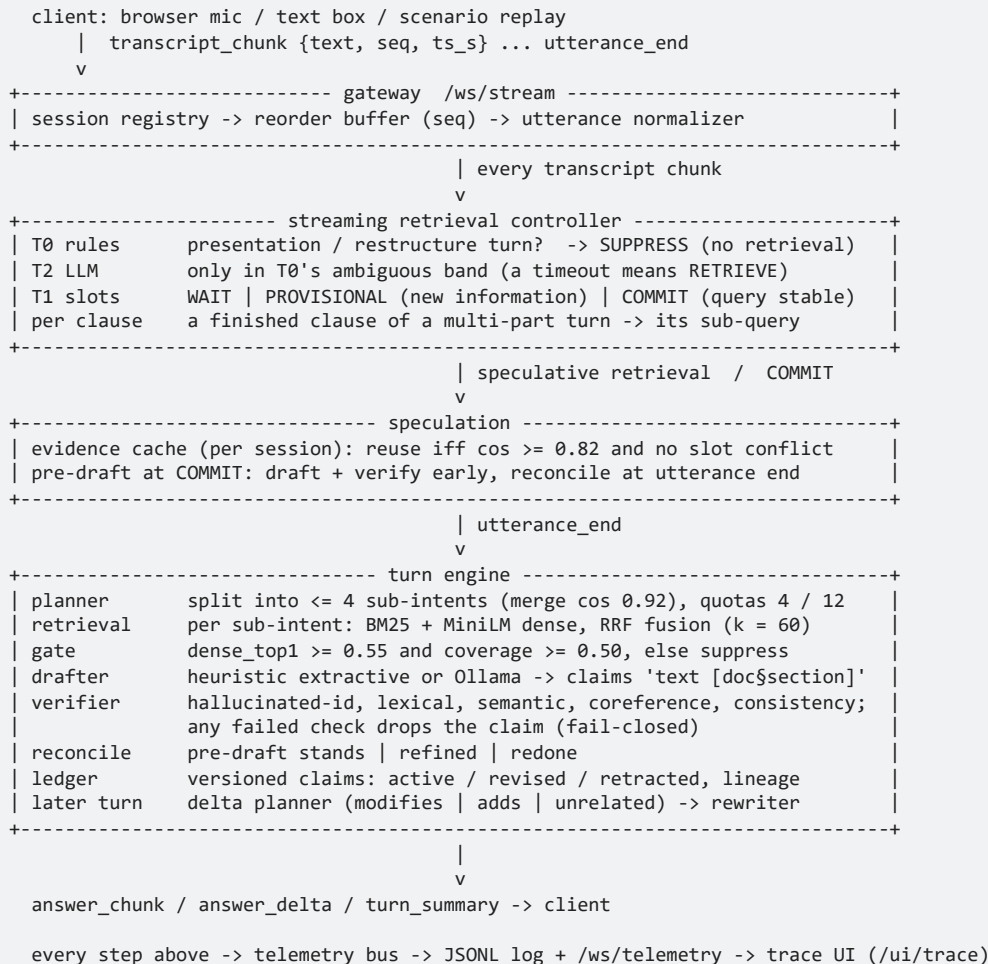
Streaming Live RAG: grounded answers over a fixed corpus, with retrieval that starts while the user is still speaking.

Every number here comes from the frozen configuration (`config.yaml`, `frozen: true`, `cfg_hash` `e3745a51d783adef`) replayed on the test split. The sources are `out/summary.json` and `out/final_experiments.json`, and `benchmark_report.md` has the full tables.

1. Problem

A voice or chat client streams a user's utterance as transcript chunks. A request-response RAG system waits for the utterance to end, then retrieves, drafts and verifies, so the user waits for all three. SL-RAG starts retrieving as soon as the partial transcript carries enough meaning. It splits multi-part questions into sub-queries and retrieves each one as its clause completes. It drafts only from retrieved evidence and refuses to emit any claim the verifier cannot ground in a cited chunk. When the user adds a constraint in a later turn, it revises only the claims that constraint affects. It does not restart the answer.

2. Data flow



Package map: gateway in `slrag/gateway`; controller in `slrag/control`; planner and delta planner in `slrag/plan`; retrieval and cache in `slrag/retrieval`; drafting, verification and the turn engine in `slrag/pipeline`; pre-draft, reconcile and rewriter in `slrag/answer`; ledger and deltas in `slrag/state`; the event contract in `slrag/contracts/events.py`; HTTP, WebSocket and the UI in `slrag/server` and `slrag/ui`.

3. Trigger logic (when to retrieve)

The controller runs on every transcript chunk and returns one decision.

- **T0, rules.** These run only when the session already has an answer. A turn made mostly of restructuring words ("summarize", "in two bullets") with no new content anchors is **SUPPRESSED**. It re-presents the existing claims without retrieving.
- **T2, LLM classifier.** It is called only in T0's ambiguous band (0.40 to 0.70), with a 0.4 s timeout. Any failure or timeout falls back to RETRIEVE, so the LLM cannot silently block retrieval.
- **T1, slots and stability.** It decides on content.
 - **PROVISIONAL:** the query built from the buffer has at least 4 content tokens and cosine below 0.86 to the last dispatched query (new information).
 - **COMMIT:** the query has been stable (cosine of at least 0.82) for 2 chunks.
 - The controller allows at most 2 PROVISIONAL and 1 COMMIT dispatches per turn.
- **Per-clause retrieval.** In a multi-intent turn, a clause that ends ("?", ".", or 2 chunks with no new content words) has its sub-query planned and dispatched at once. This is the change that lifted early retrieval from 0.789 to 0.983 on the tune split.

Thresholds were calibrated on the tune split only. The grid was `new_info_cos` × `stable_cos`; the rule was "false-trigger rate 0, then maximum early retrieval". The values were frozen before the single test-split measurement.

4. Decomposition strategy

The planner splits an utterance into at most 4 sub-intents. It works in three steps:

1. Split on clause punctuation and coordinators.
2. Split noun-phrase lists such as "the X and the Y".
3. Merge any two parts whose embeddings are more similar than 0.92 cosine.

Each sub-intent gets its own retrieval, with a quota of 4 chunks per sub-intent and 12 per turn. It gets its own sufficiency decision and its own claims. One unanswerable part therefore suppresses only itself: the other parts are still answered, and the user is told which part could not be answered.

5. Provenance and grounding

- **Ids.** Chunk ids are stable `doc_id$section_id` strings. The drafter may only cite ids from the evidence it was given.
- **The verifier is fail-closed.** A claim is emitted only if all five checks pass:
 - the cited id was in the evidence (hallucinated-id);
 - lexical overlap with the cited text is at least 0.30;
 - similarity of at least 0.65 to the cited text or its best-matching sentence (hashed n-gram vectors, a surface-overlap measure; the retrieval model is not used here);
 - no unresolved coreference;
 - no contradiction with already-verified claims.
- **The ledger.** Every claim is versioned in the claim ledger, together with its cites and history. A later constraint can keep, revise or retract a claim. Unaffected claims keep their hash: preservation is 1.0 on the test split.
- **Telemetry.** Every event (`controller_decision`, `speculative_retrieval`, `verification`, `answer_chunk`, ...) carries the session id, the turn id and the `cfg_hash`. The trace UI and the metrics are computed from this telemetry, nothing else. Trace coverage is 1.0.

6. Trade-offs and failure-mode mitigations

Decision	Benefit	Cost / risk	Mitigation
Speculative retrieval before utterance end	Test TTFT p50 388 ms vs 485 ms (B1); early retrieval 0.950	Wasted speculation 0.223; cost per turn $2.3 \times B1$	Evidence cache reuse (hit rate 0.458); max 2 PROVISIONAL per turn
Pre-draft at COMMIT (A3 full)	Draft ready earlier when the query is stable	No TTFT gain over retrieval-only on this split (388 vs 385 ms); 63% of pre-draft tokens wasted	Can be switched off: <code>speculation.mode: retrieval_only</code>
Fail-closed verifier	Groundedness 1.000, 0 hallucinated ids	Untested against a generative drafter: the extractive stand-in only copies evidence sentences	Sentence-level semantic check so short true claims are not diluted by long chunks
Delta refinement on late details	56% fewer retrieval calls than restarting	$3.5 \times$ more tokens than restarting; two JSON calls whose prompts carry claims and evidence	JSON schemas removed from the prompts (-18.1% tokens); benchmark report, failure 3
Hybrid RRF retrieval (BM25 + all-MiniLM-L6-v2)	recall@10 1.000 vs 0.968 dense-only and 0.989 BM25-only	MiniLM model load at startup (CPU); torch in the image	Model fetched once (<code>scripts/fetch_models.py</code>), loaded offline
Heuristic LLM backend by default	Deterministic, no GPU, reproducible numbers	Stand-in drafting quality	<code>llm.backend: ollama</code> for a real model; falls back per call if it is unreachable

Failure handling:

- The LLM client has a timeout and a circuit breaker. Every LLM call falls back to the heuristic path, so a turn never fails because Ollama is down.
- The reorder buffer drops stale sequence numbers and flushes at `utterance_end`.
- Sessions are isolated, and idle sessions are evicted after 30 minutes. This is checked by `scripts/compliance_audit.py`.

7. Component boundaries

The turn engine talks to duck-typed adapters (`slrag/replay/baselines.py`), so each stage can be swapped without touching the engine:

Stage	Adapter	Interface
retrieval	<code>RetrieverAdapter</code>	<code>retrieve(query) -> [SearchResult]</code>
sufficiency	<code>GateAdapter</code>	<code>evaluate(query, chunks) -> GateResult</code>
drafting	<code>DrafterAdapter</code>	<code>draft(query, chunks) -> "claim [cite]. ..."</code>
verification	<code>VerifierAdapter</code>	<code>verify(draft, chunks, turn_id) -> VerificationResult</code>
state	<code>ClaimLedger</code>	claims, versions, lineage

The **event contract** is defined as Pydantic models in `slrag/contracts/events.py`. Inbound messages on `/ws/stream` are `transcript_chunk`, `utterance_end` and `session_end`. Outbound messages are `answer_chunk`, `answer_delta` and `turn_summary`, and all internal events are broadcast on `/ws/telemetry`.

8. Deployment

- **Packaging.** `docker-compose.yml` defines two services:
 - `app`: Python 3.12 slim, a non-root user, uvicorn on :8000, with a healthcheck on `/health`.
 - `ollama`: an optional model server with a pinned image and a named model volume.

The app image installs CPU-only torch and downloads `all-MiniLM-L6-v2` at build time. `make model` downloads the retrieval model for local runs and pulls `qwen2.5:3b` into the Ollama volume; `make up` builds and starts everything.

- **Image contents.** The image contains only `slrag/`, `config.yaml`, `data/` and `pyproject.toml`. `.dockerignore` excludes `eval/`, `tests/`, `docs/`, `out/` and `scripts/`. `eval/` is mounted read-only at run time, only for the UI's scenario picker.
- **Network.** The engine makes network calls only to the configured LLM URL, and the retrieval model loads with `local_files_only=True`. Both are checked by the audit.
- **Auth.** `/ui/*` and `/api/*` use HTTP Basic auth: `admin / slrag` from `config.yaml`, which you can override with `SLRAG_UI_USERNAME / SLRAG_UI_PASSWORD`. `/ws/*` and `/health` are unauthenticated.
- **Voice.** The UI's microphone uses the browser's Web Speech API, in Chrome or Edge, over localhost or HTTPS. The browser vendor's speech service transcribes the audio. The server only ever receives text.

9. Limitations (what we do not claim)

- **Two kinds of vectors.**
 - Retrieval uses sentence-transformers `all-MiniLM-L6-v2` (384-d, CPU).
 - The controller's query-stability test, the evidence-cache reuse test and the verifier's semantic check use a hashed n-gram vector, which measures surface overlap. Their thresholds were calibrated on it, so swapping in MiniLM there would need its own tune-split recalibration.
- **The coverage gate over-suppresses paraphrases.** The sufficiency gate's coverage test suppresses some answerable requests worded with instruction verbs ("summarize the ... steps"). See benchmark report, failure 1.
- **Latency is simulated.** Latency is virtual time from the replay latency model (retrieval 120 ms, draft 350 ms, ...), not hardware measurements.
- **Cost is notional.** It is computed from token estimates and the configured per-1k prices; it is not a real-dollar cost.
- **Groundedness means the verifier's rules.** It is the verifier's own score, with no human-labelled sample or external judge. It is not a general claim of "zero hallucination".
- **The test split is small.** It has 39 scenarios plus 8 late-detail sessions (55 turns), all over a synthetic 24-document corpus. Nothing here speaks to any private held-out set.
- **The default drafter is a stand-in.** It is a heuristic extractive drafter, and answers can include loosely related claims. The Ollama backend is wired in but was not part of the measured runs.
- **Docker packaging is untested.** The Compose and Docker files were not exercised on a clean machine. The development machine has no Docker.